Brain Tumour MRI Classifier: 4 classes (PyTorch)

Classes: **glioma, meningioma, pituitary, no tumour**. After training, the same predictions are also collapsed into a *tumour vs no tumour* detector score.

Data flow: `Training/` is split into train + validation (used for tuning and early stopping); `Testing/` is touched **once**, at the very end.

```
!pip install -q kagglehub
```

```
import random
from pathlib import Path

import kagglehub
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, models, transforms

# ---- Config ----
SEED       = 42
IMG_SIZE   = 128
BATCH_SIZE = 32
MODEL      = "cnn"   # "cnn" = small net from scratch | "resnet18" = pretrained, fine-tuned
EPOCHS     = 40      # upper bound; early stopping usually ends sooner
LR         = {"cnn": 1e-3, "resnet18": 3e-4}[MODEL]   # pretrained weights need a gentler LR
PATIENCE   = 7       # epochs without val-loss improvement before stopping
VAL_FRAC   = 0.15
CKPT       = f"best_{MODEL}_4class.pt"

def set_seed(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)

set_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cuda
```

1. Data

```
root = Path(kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset"))
train_dir, test_dir = root / "Training", root / "Testing"
assert train_dir.is_dir() and test_dir.is_dir(), f"Unexpected dataset layout in {root}"

norm = transforms.Normalize([0.5], [0.5])
base = [transforms.Grayscale(1), transforms.Resize((IMG_SIZE, IMG_SIZE))]
train_tf = transforms.Compose(base + [transforms.RandomHorizontalFlip(),
                                      transforms.RandomRotation(10),
                                      transforms.ToTensor(), norm])
eval_tf  = transforms.Compose(base + [transforms.ToTensor(), norm])

# Two views of the same training folder: augmented for training, clean for validation
train_full = datasets.ImageFolder(train_dir, train_tf)
val_full   = datasets.ImageFolder(train_dir, eval_tf)
test_ds    = datasets.ImageFolder(test_dir,  eval_tf)

class_names = train_full.classes
NUM_CLASSES = len(class_names)
assert class_names == test_ds.classes, "Training/Testing class folders differ"
print("Classes:", train_full.class_to_idx)

# Stratified split keeps every class represented in both train and validation
train_idx, val_idx = train_test_split(
    np.arange(len(train_full)), test_size=VAL_FRAC,
    stratify=train_full.targets, random_state=SEED)

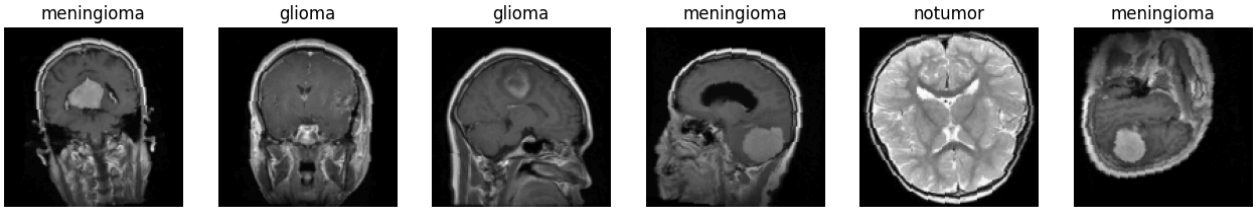
train_ds = Subset(train_full, train_idx)
val_ds   = Subset(val_full,   val_idx)
```

```
loader_kw = dict(batch_size=BATCH_SIZE, num_workers=2, pin_memory=device.type == "cuda")
train_loader = DataLoader(train_ds, shuffle=True,  generator=torch.Generator().manual_seed(SEED), **loader_kw)
val_loader   = DataLoader(val_ds,   shuffle=False, **loader_kw)
test_loader  = DataLoader(test_ds,  shuffle=False, **loader_kw)

print(f"train {len(train_ds)} | val {len(val_ds)} | test {len(test_ds)}")
print("Train class counts:", dict(zip(class_names, np.bincount(np.array(train_full.targets)[train_idx]))))
```

```
Using Colab cache for faster access to the 'brain-tumor-mri-dataset' dataset.
Classes: {'glioma': 0, 'meningioma': 1, 'notumor': 2, 'pituitary': 3}
train 4760 | val 840 | test 1600
Train class counts: {'glioma': np.int64(1190), 'meningioma': np.int64(1190), 'notumor': np.int64(1190), 'pituitar
```

```
# Quick look at one training batch
images, labels = next(iter(train_loader))
fig, axes = plt.subplots(1, 6, figsize=(16, 3))
for ax, img, lab in zip(axes, images, labels):
    ax.imshow((img[0] * 0.5 + 0.5).numpy(), cmap="gray")   # undo normalisation
    ax.set_title(class_names[lab]); ax.axis("off")
plt.show()
```



2. Model

Set `MODEL` in the config cell:

- `"cnn"`: small net from scratch, 4 × (Conv → BatchNorm → ReLU → MaxPool), global average pooling, one **logit per class**
- `"resnet18"`: ImageNet-pretrained ResNet18; the first conv is collapsed to 1 channel (weights summed over RGB) so the grayscale pipeline stays unchanged

```
def block(c_in, c_out):
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
        nn.BatchNorm2d(c_out), nn.ReLU(inplace=True), nn.MaxPool2d(2))

class CNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(block(1, 16), block(16, 32), block(32, 64), block(64, 128),
                                      nn.AdaptiveAvgPool2d(1))
        self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, num_classes))

    def forward(self, x):
        return self.classifier(self.features(x))   # raw logits, shape (B, num_classes)

def build_model(name, num_classes):
    if name == "cnn":
        return CNN(num_classes)
    if name == "resnet18":
        net = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
        w = net.conv1.weight.data.sum(dim=1, keepdim=True)          # RGB -> grayscale
        net.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
        net.conv1.weight.data = w
        net.fc = nn.Sequential(nn.Dropout(0.3), nn.Linear(net.fc.in_features, num_classes))
        return net
    raise ValueError(f"Unknown MODEL: {name}")

model = build_model(MODEL, NUM_CLASSES).to(device)
print(f"{MODEL}: {sum(p.numel() for p in model.parameters()):,} parameters")
```

```
cnn: 97,908 parameters
```

3. Training

- `CrossEntropyLoss` (softmax is built in, so the model outputs raw logits)
- LR scheduler + early stopping on validation loss; the best weights are saved and reloaded

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, factor=0.5, patience=2)

def run_epoch(loader, train):
    model.train(train)
    total_loss = correct = n = 0
    with torch.set_grad_enabled(train):
        for x, y in loader:
            x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
            logits = model(x)
            loss = criterion(logits, y)
            if train:
                optimizer.zero_grad(set_to_none=True)
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * len(y)
            correct    += (logits.argmax(1) == y).sum().item()
            n += len(y)
    return total_loss / n, correct / n

history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_val, stale = float("inf"), 0

for epoch in range(1, EPOCHS + 1):
    tr_loss, tr_acc = run_epoch(train_loader, train=True)
    va_loss, va_acc = run_epoch(val_loader,   train=False)
    scheduler.step(va_loss)
    for k, v in zip(history, (tr_loss, va_loss, tr_acc, va_acc)):
        history[k].append(v)

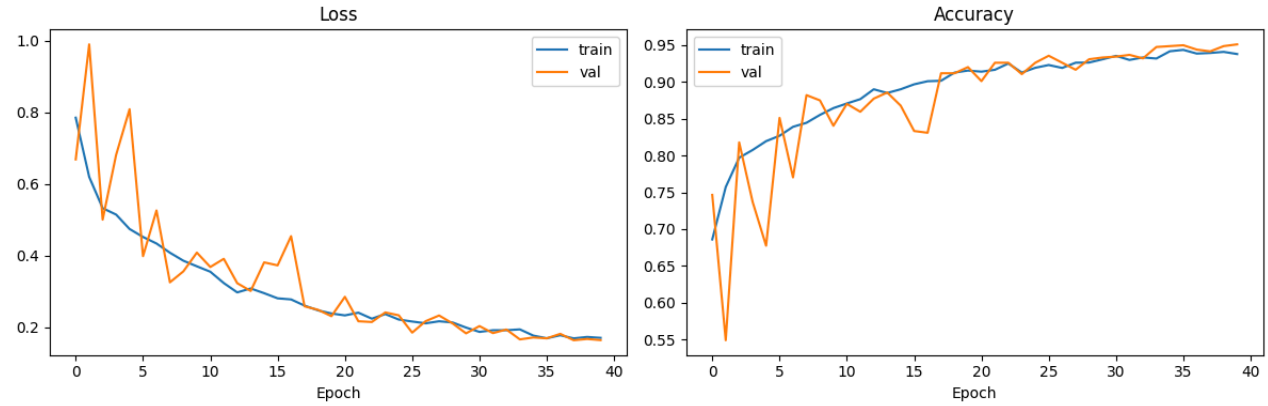
    improved = va_loss < best_val
    if improved:
        best_val, stale = va_loss, 0
        torch.save(model.state_dict(), CKPT)
    else:
        stale += 1

    print(f"Epoch {epoch:02d} | train loss {tr_loss:.4f} acc {tr_acc:.3f} | "
          f"val loss {va_loss:.4f} acc {va_acc:.3f}{'  *' if improved else ''}")
    if stale >= PATIENCE:
        print(f"Early stopping (no val-loss improvement for {PATIENCE} epochs)")
        break
```

```
Epoch 01 | train loss 0.7845 acc 0.686 | val loss 0.6681 acc 0.746  *
Epoch 02 | train loss 0.6193 acc 0.757 | val loss 0.9891 acc 0.549
Epoch 03 | train loss 0.5320 acc 0.797 | val loss 0.5001 acc 0.818  *
Epoch 04 | train loss 0.5141 acc 0.808 | val loss 0.6806 acc 0.737
Epoch 05 | train loss 0.4739 acc 0.820 | val loss 0.8082 acc 0.677
Epoch 06 | train loss 0.4516 acc 0.827 | val loss 0.3981 acc 0.851  *
Epoch 07 | train loss 0.4334 acc 0.839 | val loss 0.5256 acc 0.770
Epoch 08 | train loss 0.4077 acc 0.845 | val loss 0.3251 acc 0.882  *
Epoch 09 | train loss 0.3853 acc 0.855 | val loss 0.3557 acc 0.875
Epoch 10 | train loss 0.3698 acc 0.864 | val loss 0.4080 acc 0.840
Epoch 11 | train loss 0.3546 acc 0.871 | val loss 0.3678 acc 0.870
Epoch 12 | train loss 0.3230 acc 0.877 | val loss 0.3907 acc 0.860
Epoch 13 | train loss 0.2969 acc 0.890 | val loss 0.3227 acc 0.877  *
Epoch 14 | train loss 0.3079 acc 0.885 | val loss 0.3009 acc 0.886  *
Epoch 15 | train loss 0.2948 acc 0.890 | val loss 0.3807 acc 0.868
Epoch 16 | train loss 0.2804 acc 0.897 | val loss 0.3724 acc 0.833
Epoch 17 | train loss 0.2774 acc 0.901 | val loss 0.4538 acc 0.831
Epoch 18 | train loss 0.2601 acc 0.902 | val loss 0.2581 acc 0.912  *
Epoch 19 | train loss 0.2469 acc 0.913 | val loss 0.2484 acc 0.912  *
Epoch 20 | train loss 0.2381 acc 0.916 | val loss 0.2306 acc 0.920  *
Epoch 21 | train loss 0.2328 acc 0.914 | val loss 0.2846 acc 0.901
Epoch 22 | train loss 0.2403 acc 0.917 | val loss 0.2164 acc 0.926  *
Epoch 23 | train loss 0.2236 acc 0.925 | val loss 0.2143 acc 0.926  *
Epoch 24 | train loss 0.2365 acc 0.913 | val loss 0.2411 acc 0.911
Epoch 25 | train loss 0.2212 acc 0.919 | val loss 0.2330 acc 0.926
Epoch 26 | train loss 0.2157 acc 0.923 | val loss 0.1847 acc 0.936  *
Epoch 27 | train loss 0.2107 acc 0.919 | val loss 0.2167 acc 0.926
Epoch 28 | train loss 0.2164 acc 0.926 | val loss 0.2325 acc 0.917
Epoch 29 | train loss 0.2130 acc 0.926 | val loss 0.2104 acc 0.931
Epoch 30 | train loss 0.1988 acc 0.931 | val loss 0.1826 acc 0.933  *
Epoch 31 | train loss 0.1867 acc 0.936 | val loss 0.2029 acc 0.935
Epoch 32 | train loss 0.1914 acc 0.930 | val loss 0.1833 acc 0.937
Epoch 33 | train loss 0.1914 acc 0.933 | val loss 0.1933 acc 0.932
Epoch 34 | train loss 0.1937 acc 0.932 | val loss 0.1658 acc 0.948  *
Epoch 35 | train loss 0.1764 acc 0.942 | val loss 0.1711 acc 0.949
Epoch 36 | train loss 0.1690 acc 0.944 | val loss 0.1688 acc 0.950
Epoch 37 | train loss 0.1773 acc 0.939 | val loss 0.1813 acc 0.944
```

```
Epoch 38 | train loss 0.1687 acc 0.939 | val loss 0.1635 acc 0.942  *
Epoch 39 | train loss 0.1726 acc 0.941 | val loss 0.1670 acc 0.949
Epoch 40 | train loss 0.1701 acc 0.938 | val loss 0.1638 acc 0.951
```

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(12, 4))
a1.plot(history["train_loss"], label="train"); a1.plot(history["val_loss"], label="val")
a1.set(title="Loss", xlabel="Epoch"); a1.legend()
a2.plot(history["train_acc"], label="train"); a2.plot(history["val_acc"], label="val")
a2.set(title="Accuracy", xlabel="Epoch"); a2.legend()
plt.tight_layout(); plt.show()
```



Plateau check

Reads the training history and says whether accuracy has stalled, and if so what to change.

```
best_ep  = int(np.argmax(history["val_acc"]))
best_acc = history["val_acc"][best_ep]
tail     = history["val_acc"][-5:]
gain     = max(tail) - min(tail) if len(tail) > 1 else 0.0
gap      = history["train_acc"][best_ep] - best_acc

print(f"Best val acc {best_acc:.3f} at epoch {best_ep + 1} of {len(history['val_acc'])}")
print(f"Val-acc range over last 5 epochs: {gain:.3f} | train-val gap at best epoch: {gap:.3f}")

plateaued = gain < 0.005 or (len(history["val_acc"]) - best_ep) > 5
if not plateaued:
    print("-> Still improving. Raise EPOCHS / PATIENCE and continue.")
elif best_acc < 0.90:
    print("-> Plateau with LOW accuracy (underfitting). Set MODEL = 'resnet18'; also try IMG_SIZE = 224.")
elif gap > 0.05:
    print("-> Plateau with a large train-val gap (overfitting). Set MODEL = 'resnet18' or add stronger augmentat
else:
    print("-> Plateau at a good level with a small gap. Remaining errors are likely hard glioma/meningioma cases
```

```
Best val acc 0.951 at epoch 40 of 40
Val-acc range over last 5 epochs: 0.010 | train-val gap at best epoch: -0.013
-> Still improving. Raise EPOCHS / PATIENCE and continue.
```

4. Final evaluation on the held-out `Testing/` split (run once)

```
model.load_state_dict(torch.load(CKPT, map_location=device))
model.eval()

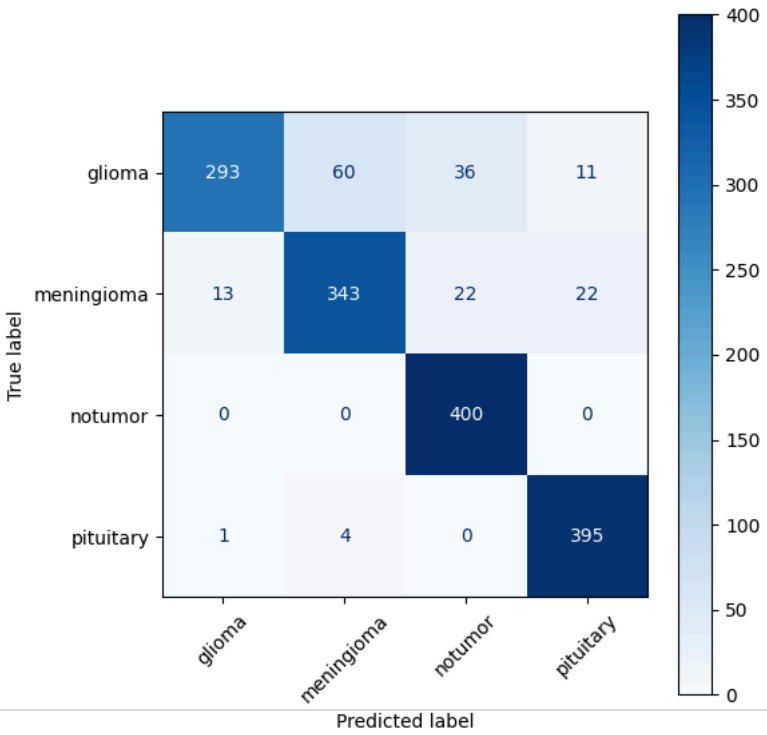
preds, y_true = [], []
with torch.no_grad():
    for x, y in test_loader:
        preds.append(model(x.to(device)).argmax(1).cpu()); y_true.append(y)
y_pred, y_true = torch.cat(preds).numpy(), torch.cat(y_true).numpy()

print(classification_report(y_true, y_pred, target_names=class_names, digits=4))
fig, ax = plt.subplots(figsize=(6, 6))
ConfusionMatrixDisplay.from_predictions(y_true, y_pred, display_labels=class_names,
                                        cmap="Blues", xticks_rotation=45, ax=ax)
plt.tight_layout(); plt.show()
```

```
              precision    recall  f1-score   support

      glioma     0.9544    0.7325    0.8289       400
  meningioma     0.8428    0.8575    0.8501       400
     notumor     0.8734    1.0000    0.9324       400
   pituitary     0.9229    0.9875    0.9541       400

    accuracy                         0.8944      1600
   macro avg     0.8984    0.8944    0.8914      1600
weighted avg     0.8984    0.8944    0.8914      1600
```



Tumour vs no-tumour view

The same test predictions, collapsed into the original detector question: is *any* tumour present? Confusing glioma with meningioma counts as correct here, since a tumour was still detected.

```
healthy = train_full.class_to_idx["notumor"]
det_true, det_pred = (y_true != healthy).astype(int), (y_pred != healthy).astype(int)
print(classification_report(det_true, det_pred, target_names=["No tumour", "Tumour"], digits=4))
```

```
              precision    recall  f1-score   support

   No tumour     0.8734    1.0000    0.9324       400
      Tumour     1.0000    0.9517    0.9752      1200

    accuracy                         0.9637      1600
   macro avg     0.9367    0.9758    0.9538      1600
weighted avg     0.9683    0.9637    0.9645      1600
```